BACKEND SYSTEM DESIGN ASSIGNMENT

Multi-Tenant Text-to-SQL and Document Chat Platform

Backend Architecture, Database Design, Security Controls, and API Specification

Work mode	Individual assignment
Deadline	4 days from the date the assignment is issued
Submission	Backend source code, database schema, setup instructions, and API documentation

Important: Each student must design and implement the project independently. Shared implementations or copied submissions are not permitted.

1. Assignment Objective

Design and implement a secure, scalable backend platform that allows authenticated users to connect live business databases, upload files, and chat with both sources through one conversational interface. The platform must support database-only questions, document-only questions, and hybrid questions that combine structured database results with evidence from uploaded documents.

- Add database connections at runtime without changing application source code.
- Discover and cache database schemas while keeping business data in the source database.
- Upload and process PDF, Word, Excel, CSV, and text files.
- Generate safe SQL from natural-language questions.
- Retrieve relevant document chunks through vector search.
- Combine SQL results and document evidence into one grounded answer.
- Apply tenant, role, table, column, and row-level permissions.
- Store conversations, citations, query executions, and audit records.

2. Required System Capabilities

Capability	Required behavior	Expected result
Live database connection	A tenant administrator submits connection details and tests the connection at runtime.	The connection is encrypted, validated, and available for authorized chats.
Schema discovery	The system reads schemas, tables, columns, keys, and relationships.	Only approved metadata is cached in the application database.
File ingestion	Users upload supported files into a knowledge base.	Files are parsed, chunked, embedded, and indexed.
Text-to-SQL	The system selects an allowed schema and generates SQL.	Validated SQL is executed using controlled credentials and limits.
Document chat	The system retrieves relevant chunks from selected files.	The answer includes document citations such as file and page.
Hybrid chat	The orchestrator runs SQL and document retrieval as required.	The final answer compares or combines both sources.
Security	Permissions and backend filters are applied before	The LLM cannot bypass access controls or create its own

Capability	Required behavior	Expected result
	execution.	tenant filters.

3. High-Level Architecture

```
Client / Frontend
      |
      v
FastAPI API Gateway
      |
      +-- Authentication and Tenant Context
      +-- Connection and Schema Management
      +-- File Upload and Processing
      +-- Conversation and Streaming API
      |
      v
Chat Orchestrator / LangGraph
      |
      +-- Request Classifier
      +-- Source Selector
      +-- Database Agent
      |     +-- Schema Retriever
      |     +-- SQL Generator
      |     +-- SQL Validator
      |     +-- Query Executor
      |
      +-- Document RAG Agent
      |     +-- Query Rewriter
      |     +-- Vector Retriever
      |     +-- Evidence Reranker
      |
      +-- Hybrid Result Merger
      +-- Final Answer Generator
      |
      +-----------------------------------------+
      | Application DB | Vector DB | File Store |
      +-----------------------------------------+
                  |
                  v
            Live Customer Databases
```

The platform application database stores identities, permissions, connection metadata, schema cache, files, conversations, and audit records. Customer business data remains in each connected source database and is queried only when an authorized request is executed.

4. Recommended Backend Project Structure

```
text-to-sql-platform/
|-- app/
|   |-- main.py
|   |-- config.py
|   |-- dependencies.py
|   |-- exceptions.py
|   `-- logging_config.py
|-- api/
|   |-- router.py
```

```
|   `-- routes/
|       |-- auth.py
|       |-- tenants.py
|       |-- users.py
|       |-- database_connections.py
|       |-- database_schema.py
|       |-- files.py
|       |-- knowledge_bases.py
|       |-- conversations.py
|       |-- chat.py
|       |-- permissions.py
|       `-- health.py
|-- core/
|   |-- security.py
|   |-- encryption.py
|   |-- permissions.py
|   |-- constants.py
|   `-- tenant_context.py
|-- models/
|   |-- tenant.py
|   |-- user.py
|   |-- role.py
|   |-- database_connection.py
|   |-- database_schema.py
|   |-- table_permission.py
|   |-- file.py
|   |-- document_chunk.py
|   |-- knowledge_base.py
|   |-- conversation.py
|   |-- message.py
|   |-- query_execution.py
|   |-- citation.py
|   `-- audit_log.py
|-- schemas/
|-- repositories/
|-- services/
|   |-- database/
|   |   |-- connection_service.py
|   |   |-- connection_tester.py
|   |   |-- schema_discovery.py
|   |   |-- metadata_cache.py
|   |   |-- query_executor.py
|   |   |-- query_validator.py
|   |   |-- dialect_resolver.py
|   |   `-- adapters/
|   |       |-- base.py
|   |       |-- postgresql.py
|   |       |-- sqlserver.py
|   |       |-- mysql.py
|   |       `-- oracle.py
|   |-- documents/
|   |   |-- upload_service.py
|   |   |-- document_processor.py
|   |   |-- chunking_service.py
|   |   |-- embedding_service.py
|   |   |-- retrieval_service.py
|   |   `-- parsers/
|   |-- chat/
|   `-- llm/
|-- agents/
|   |-- graph.py
|   |-- state.py
|   |-- nodes/
```

```
|   `-- prompts/
|-- storage/
|-- vector_store/
|-- workers/
|-- migrations/
|-- tests/
|-- scripts/
|-- .env.example
|-- alembic.ini
|-- requirements.txt
|-- Dockerfile
|-- docker-compose.yml
`-- README.md
```

5. Chat Orchestration Flow

1. Authenticate the request and resolve the active tenant and user.
2. Load the conversation, selected database connections, selected knowledge bases, and recent chat history.
3. Classify the request as general, database, document, hybrid, or clarification.
4. Resolve the tables and columns that the current user is permitted to access.
5. For database questions, retrieve the filtered schema, generate SQL, validate it, inject backend row filters, and execute it.
6. For document questions, retrieve relevant chunks from the selected knowledge bases and rerank the evidence.
7. For hybrid questions, run database and document retrieval in parallel where possible.
8. Generate a final answer from the approved outputs only.
9. Save the question, answer, SQL execution details, citations, latency, and audit event.
10. Stream the answer to the client using Server-Sent Events when requested.

```
GENERAL = "general"
DATABASE = "database"
DOCUMENT = "document"
HYBRID = "hybrid"
CLARIFICATION = "clarification"
```

6. Agent Design Rule

Do not create one permanent agent for every database table. Build one generic database agent that receives a request-specific, permission-filtered schema. This keeps the platform general across different tenants and industries.

```
allowed_schema = {
   "customers": {
      "access": "read",
      "columns": ["id", "name", "country"]
   },
   "orders": {
      "access": "read_write",
      "columns": ["id", "customer_id", "amount", "status"]
   }
}
```

7. Application Database Design

Use PostgreSQL for the platform database. The following schema is a reference design. Students may extend it, but the core entities and controls must remain.

7.1 Tenants and Users

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE tenants (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(200) NOT NULL,
    code VARCHAR(100) NOT NULL UNIQUE,
    status VARCHAR(30) NOT NULL DEFAULT 'active',
    settings JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    email VARCHAR(255) NOT NULL,
    full_name VARCHAR(255),
    password_hash TEXT,
    status VARCHAR(30) NOT NULL DEFAULT 'active',
    is_tenant_admin BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_users_tenant_email UNIQUE (tenant_id, email)
);
CREATE INDEX idx_users_tenant_id ON users(tenant_id);
```

7.2 Roles

```
CREATE TABLE roles (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_roles_tenant_name UNIQUE (tenant_id, name)
);

CREATE TABLE user_roles (
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role_id UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (user_id, role_id)
);
```

7.3 Live Database Connections

```
CREATE TABLE database_connections (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    name VARCHAR(200) NOT NULL,
    database_type VARCHAR(50) NOT NULL,
    host VARCHAR(255),
    port INTEGER,
    database_name VARCHAR(255),
```

```
    username VARCHAR(255),
    encrypted_password TEXT,
    encrypted_connection_string TEXT,
    ssl_enabled BOOLEAN NOT NULL DEFAULT FALSE,
    ssl_settings JSONB NOT NULL DEFAULT '{}'::jsonb,
    connection_options JSONB NOT NULL DEFAULT '{}'::jsonb,
    status VARCHAR(30) NOT NULL DEFAULT 'pending',
    last_tested_at TIMESTAMPTZ,
    last_test_message TEXT,
    schema_sync_status VARCHAR(30) DEFAULT 'pending',
    last_schema_sync_at TIMESTAMPTZ,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_database_connection_name UNIQUE (tenant_id, name)
);
CREATE INDEX idx_database_connections_tenant ON database_connections(tenant_id);
```

7.4 Cached Database Metadata

```
CREATE TABLE database_schemas (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    connection_id UUID NOT NULL REFERENCES database_connections(id) ON DELETE CASCADE,
    schema_name VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_database_schema UNIQUE (connection_id, schema_name)
);

CREATE TABLE database_tables (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    connection_id UUID NOT NULL REFERENCES database_connections(id) ON DELETE CASCADE,
    schema_id UUID REFERENCES database_schemas(id) ON DELETE CASCADE,
    table_name VARCHAR(255) NOT NULL,
    table_type VARCHAR(50) NOT NULL DEFAULT 'table',
    description TEXT,
    estimated_row_count BIGINT,
    primary_key_columns JSONB NOT NULL DEFAULT '[]'::jsonb,
    is_enabled BOOLEAN NOT NULL DEFAULT TRUE,
    is_sensitive BOOLEAN NOT NULL DEFAULT FALSE,
    metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_database_table UNIQUE (connection_id, schema_id, table_name)
);

CREATE TABLE database_columns (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    table_id UUID NOT NULL REFERENCES database_tables(id) ON DELETE CASCADE,
    column_name VARCHAR(255) NOT NULL,
    data_type VARCHAR(100) NOT NULL,
    ordinal_position INTEGER,
    is_nullable BOOLEAN,
    is_primary_key BOOLEAN NOT NULL DEFAULT FALSE,
    is_foreign_key BOOLEAN NOT NULL DEFAULT FALSE,
    is_sensitive BOOLEAN NOT NULL DEFAULT FALSE,
    referenced_schema VARCHAR(255),
    referenced_table VARCHAR(255),
    referenced_column VARCHAR(255),
```

```
    description TEXT,
    sample_values JSONB NOT NULL DEFAULT '[]'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_database_column UNIQUE (table_id, column_name)
);
```

7.5 Table and Column Permissions

```
CREATE TABLE table_permissions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    role_id UUID REFERENCES roles(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    connection_id UUID NOT NULL REFERENCES database_connections(id) ON DELETE CASCADE,
    table_id UUID NOT NULL REFERENCES database_tables(id) ON DELETE CASCADE,
    can_read BOOLEAN NOT NULL DEFAULT TRUE,
    can_insert BOOLEAN NOT NULL DEFAULT FALSE,
    can_update BOOLEAN NOT NULL DEFAULT FALSE,
    can_delete BOOLEAN NOT NULL DEFAULT FALSE,
    row_filter JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_permission_subject CHECK (
        (role_id IS NOT NULL AND user_id IS NULL)
        OR (role_id IS NULL AND user_id IS NOT NULL)
    )
);

CREATE TABLE column_permissions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    table_permission_id UUID NOT NULL REFERENCES table_permissions(id) ON DELETE CASCADE,
    column_id UUID NOT NULL REFERENCES database_columns(id) ON DELETE CASCADE,
    can_read BOOLEAN NOT NULL DEFAULT TRUE,
    can_filter BOOLEAN NOT NULL DEFAULT TRUE,
    can_aggregate BOOLEAN NOT NULL DEFAULT TRUE,
    mask_type VARCHAR(50),
    CONSTRAINT uq_column_permission UNIQUE (table_permission_id, column_id)
);
```

7.6 Knowledge Bases and Files

```
CREATE TABLE knowledge_bases (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    embedding_model VARCHAR(255),
    chunking_config JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_knowledge_base_name UNIQUE (tenant_id, name)
);

CREATE TABLE files (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    knowledge_base_id UUID REFERENCES knowledge_bases(id) ON DELETE SET NULL,
    uploaded_by UUID REFERENCES users(id) ON DELETE SET NULL,
    original_name VARCHAR(500) NOT NULL,
    stored_name VARCHAR(500) NOT NULL,
    storage_path TEXT NOT NULL,
    mime_type VARCHAR(255),
```

```
    extension VARCHAR(30),
    file_size_bytes BIGINT,
    checksum VARCHAR(128),
    processing_status VARCHAR(30) NOT NULL DEFAULT 'pending',
    processing_error TEXT,
    page_count INTEGER,
    extracted_text_length BIGINT,
    metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    processed_at TIMESTAMPTZ
);
```

7.7 Document Chunks and Embeddings

```
CREATE TABLE document_chunks (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    knowledge_base_id UUID NOT NULL REFERENCES knowledge_bases(id) ON DELETE CASCADE,
    file_id UUID NOT NULL REFERENCES files(id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    content_hash VARCHAR(128),
    page_number INTEGER,
    section_title TEXT,
    token_count INTEGER,
    metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
    embedding VECTOR(1024),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_document_chunk UNIQUE (file_id, chunk_index)
);
```

7.8 Conversations and Messages

```
CREATE TABLE conversations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(500),
    status VARCHAR(30) NOT NULL DEFAULT 'active',
    active_connection_ids JSONB NOT NULL DEFAULT '[]'::jsonb,
    active_knowledge_base_ids JSONB NOT NULL DEFAULT '[]'::jsonb,
    settings JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_message_at TIMESTAMPTZ
);

CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    parent_message_id UUID REFERENCES messages(id) ON DELETE SET NULL,
    role VARCHAR(30) NOT NULL,
    message_type VARCHAR(30) NOT NULL DEFAULT 'text',
    content TEXT NOT NULL,
    structured_content JSONB,
    detected_intent VARCHAR(50),
    selected_sources JSONB NOT NULL DEFAULT '[]'::jsonb,
    model_name VARCHAR(255),
    prompt_tokens INTEGER,
    completion_tokens INTEGER,
    latency_ms INTEGER,
    status VARCHAR(30) NOT NULL DEFAULT 'completed',
```

```
    error_message TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.9 Query Executions

```
CREATE TABLE query_executions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    conversation_id UUID REFERENCES conversations(id) ON DELETE SET NULL,
    message_id UUID REFERENCES messages(id) ON DELETE SET NULL,
    connection_id UUID NOT NULL REFERENCES database_connections(id) ON DELETE CASCADE,
    generated_sql TEXT NOT NULL,
    normalized_sql TEXT,
    query_type VARCHAR(30),
    validation_status VARCHAR(30) NOT NULL,
    validation_errors JSONB NOT NULL DEFAULT '[]'::jsonb,
    applied_row_filters JSONB NOT NULL DEFAULT '{}'::jsonb,
    referenced_tables JSONB NOT NULL DEFAULT '[]'::jsonb,
    referenced_columns JSONB NOT NULL DEFAULT '[]'::jsonb,
    execution_status VARCHAR(30),
    execution_time_ms INTEGER,
    returned_row_count INTEGER,
    result_preview JSONB,
    error_code VARCHAR(100),
    error_message TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.10 Citations and Audit Logs

```
CREATE TABLE message_citations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,
    citation_type VARCHAR(30) NOT NULL,
    file_id UUID REFERENCES files(id) ON DELETE SET NULL,
    chunk_id UUID REFERENCES document_chunks(id) ON DELETE SET NULL,
    query_execution_id UUID REFERENCES query_executions(id) ON DELETE SET NULL,
    title TEXT,
    source_reference TEXT,
    page_number INTEGER,
    relevance_score NUMERIC(8,6),
    metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID REFERENCES tenants(id) ON DELETE SET NULL,
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(100) NOT NULL,
    resource_type VARCHAR(100),
    resource_id UUID,
    ip_address INET,
    user_agent TEXT,
    request_id VARCHAR(100),
    details JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

8. Required API Endpoints

```
POST   /api/auth/login
POST   /api/auth/refresh
GET    /api/auth/me

POST   /api/database-connections
GET    /api/database-connections
GET    /api/database-connections/{id}
PUT    /api/database-connections/{id}
DELETE /api/database-connections/{id}
POST   /api/database-connections/{id}/test
POST   /api/database-connections/{id}/sync-schema
GET    /api/database-connections/{id}/schemas
GET    /api/database-connections/{id}/tables

POST   /api/files/upload
GET    /api/files
GET    /api/files/{id}
DELETE /api/files/{id}
POST   /api/files/{id}/reprocess

POST   /api/knowledge-bases
GET    /api/knowledge-bases
POST   /api/knowledge-bases/{id}/files

POST   /api/conversations
GET    /api/conversations
GET    /api/conversations/{id}
DELETE /api/conversations/{id}

POST   /api/chat
POST   /api/chat/stream
GET    /api/messages/{id}/citations
GET    /api/messages/{id}/sql
```

9. Chat Request and Response Contract

Request example

```
{
 "conversation_id": "642640ab-e08c-4166-bf86-a270617fa5ef",
 "message": "Compare the total invoice value with the contract values mentioned in the uploaded files.",
 "database_connection_ids": [
  "c74663c6-d216-42a9-acf1-9244ccb709a6"
 ],
 "knowledge_base_ids": [
  "86f4e208-b048-4916-a004-42df284c99c6"
 ],
 "stream": true
}
```

Response example

```
{
 "message_id": "857cf595-d4cb-4281-9859-34d87cb4028c",
 "answer": "The invoices total 8.4 million EGP, while the uploaded contract indicates an approved value of 9 million EGP.",
 "intent": "hybrid",
 "sources_used": ["database", "documents"],
 "sql": {
  "query_execution_id": "e18443ba-00ac-44f1-b302-dd40329db01e",
```

```
  "query": "SELECT SUM(invoice_value) AS total_invoice_value FROM invoices",
  "row_count": 1
 },
 "citations": [
  {"type": "document", "file_name": "contract.pdf", "page": 14},
  {"type": "database", "table": "invoices"}
 ]
}
```

10. Mandatory SQL Security Controls

11. Expose only the tables and columns permitted for the current user.
12. Use a SQL parser such as SQLGlot; do not rely only on keyword matching.
13. Block multiple statements and SQL comments.
14. Block system schemas and administrative functions.
15. Allow read-only queries by default.
16. Block DDL and destructive DML unless a separate approved workflow exists.
17. Apply row limits, execution timeouts, and result-size limits.
18. Inject tenant and row-level filters from backend authorization rules.
19. Never allow the LLM to create, remove, or modify mandatory security filters.
20. Use read-only source database credentials for normal chat.
21. Log generated SQL, normalized SQL, validation decisions, execution status, and referenced objects.
22. Mask sensitive columns in prompts, query results, logs, and final answers.

Allowed by default

```
SELECT
WITH
EXPLAIN
```

Blocked by default

```
DROP
TRUNCATE
ALTER
CREATE
GRANT
REVOKE
EXEC
CALL
COPY
ATTACH
DETACH
```

11. Recommended Technology Stack

Layer	Recommended technology
Backend	FastAPI
Application database	PostgreSQL

Layer	Recommended technology
ORM and migrations	SQLAlchemy 2 and Alembic
Agent orchestration	LangGraph
Queue	Celery or Dramatiq
Cache	Redis
Vector store	pgvector or Qdrant
Object storage	MinIO, S3, or Azure Blob Storage
SQL validation	SQLGlot plus database-specific rules
Document parsing	Docling and format-specific parsers
Streaming	Server-Sent Events
Monitoring	Prometheus, Grafana, and OpenTelemetry

12. Suggested Deployment Services

```
api
worker
postgres
redis
qdrant
minio
prometheus
grafana
```

13. Four-Day Implementation Plan

Day	Main work	Expected checkpoint
Day 1	Project setup, PostgreSQL schema, authentication, tenant context, connection CRUD, credential encryption, and connection testing.	User can sign in, create a database connection, test it, and store it securely.
Day 2	Schema discovery, metadata cache, table and column permission APIs, SQL generation, SQL parsing, validation, and safe query execution.	A permitted natural-language database question returns validated query results.
Day 3	File upload, object storage, document parsing, chunking, embeddings, vector retrieval, knowledge-base management, and document citations.	A user can upload a file and ask a grounded question about it.
Day 4	Hybrid orchestration, streaming chat, conversation persistence, audit logs, integration tests, security tests, Docker setup, and documentation.	A hybrid question uses both database and document evidence; the project is packaged for submission.

14. Required Deliverables

- Complete backend source code with a clear modular structure.
- Database migration files or a complete schema.sql file.
- A .env.example file without real credentials or secrets.
- Dockerfile and docker-compose.yml for required infrastructure.
- README containing setup, migration, run, test, and API usage instructions.
- OpenAPI documentation and example requests.
- Unit tests for validators and permission logic.
- Integration tests for connection testing, schema discovery, file processing, database chat, document chat, and hybrid chat.
- Security tests demonstrating that unauthorized tables, columns, rows, and destructive SQL are blocked.
- A short architecture explanation and a diagram.

15. Acceptance Criteria

Criterion	Minimum acceptable result
Multi-tenancy	Data and resources from one tenant cannot be accessed by another tenant.
Runtime connections	A new supported database can be connected and used without a source-code change.
Permissions	The prompt and execution layer expose only approved schemas, tables, columns, and rows.
Safe SQL	Unsafe or unauthorized SQL is rejected before reaching the source database.
Document processing	Supported files are parsed, indexed, searchable, and cited.
Hybrid answers	The system can combine a database calculation with evidence from uploaded files.
Traceability	Every answer can be traced to SQL execution records and/or document chunks.
Reliability	Failures are handled with clear statuses and without exposing secrets or internal stack traces.
Documentation	Another developer can run the solution by following the README.

16. Individual Work Requirement

Each student must complete this assignment independently within four days. Students may consult public documentation, but the architecture decisions, implementation, tests, and written submission must be their own. Any external code or reference material used must be clearly acknowledged in the README.

17. Final Architecture Principle

```
Application Database
  Stores users, tenants, permissions, connection metadata,
  schema cache, files, conversations, citations, and audits.
```

```
Customer Databases
  Store live business data and are queried only through
  validated, permission-controlled, time-limited execution.

Vector Database
  Stores embeddings for uploaded document chunks.

Object Storage
  Stores original uploaded files and processed artifacts.
```

Do not copy all customer data into the application database. Cache only the metadata and limited approved values required for schema understanding and safe query generation.